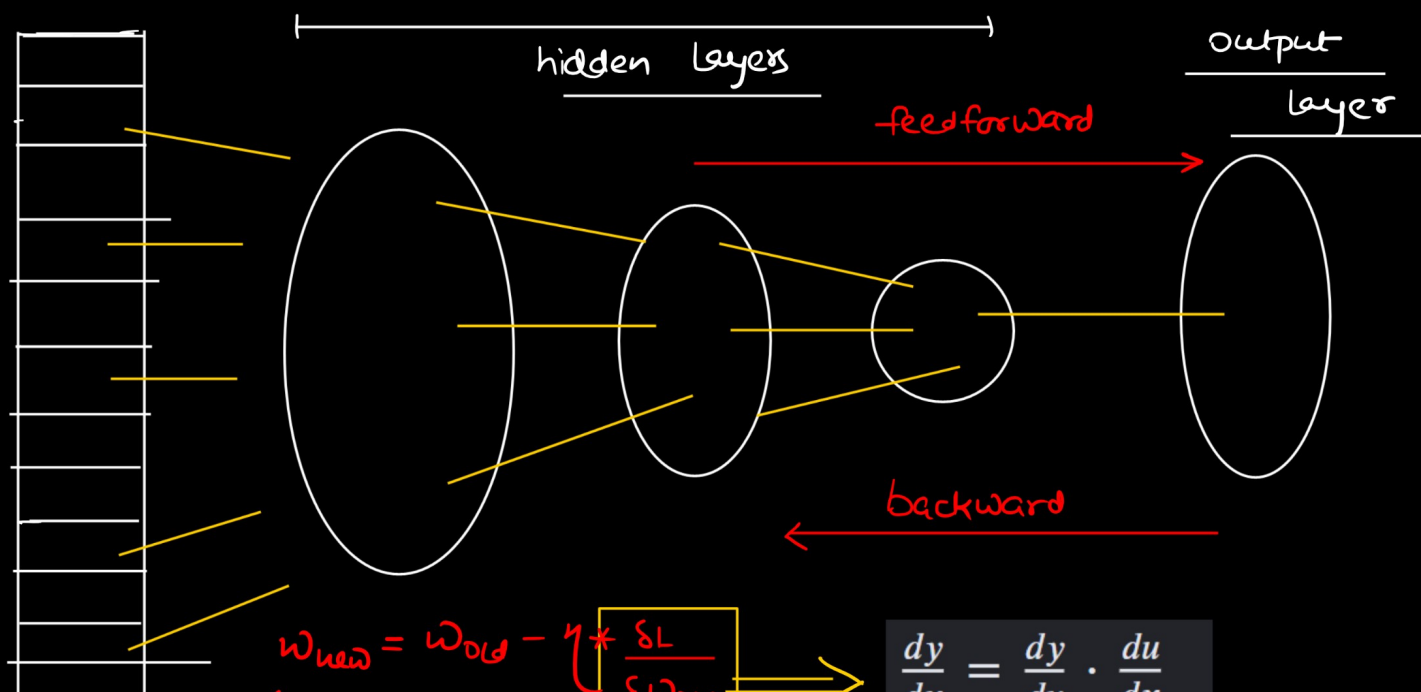


Deep Neural Networks



$$\frac{dy}{dx} = \frac{dy}{du} * \frac{du}{dx}$$

$$\frac{dy}{dx} = \frac{dy}{du} * \frac{du}{dx}$$

$w_{new} \approx w_{old}$

Learning

has

stopped

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial a_n} * \frac{\partial a_n}{\partial a_{n-1}} * \frac{\partial a_{n-1}}{\partial a_{n-2}} * \dots * \frac{\partial a_1}{\partial w_i}$$

very

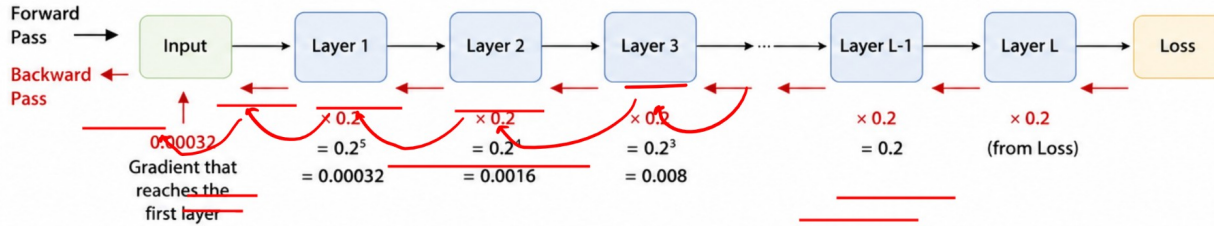
low

$$0.01 * 0.01 * 0.01 * 0.01$$

very very low

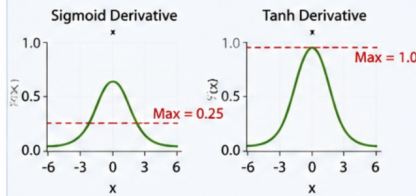
THE VANISHING GRADIENT PROBLEM

As gradients backpropagate through many layers, they get multiplied by small numbers (< 1) repeatedly, becoming extremely close to zero. Early layers learn almost nothing.



WHY DOES IT HAPPEN?

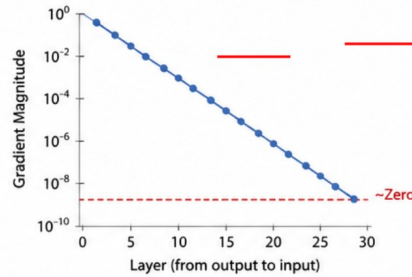
For activations like sigmoid or tanh, their derivatives are always < 1 . When many such derivatives are multiplied together, the result approaches zero.



Since derivatives are ≤ 1 , multiplying many of them shrinks the gradient exponentially.

GRADIENT SHRINKS EXPONENTIALLY

Example: multiplying by 0.2 at each layer

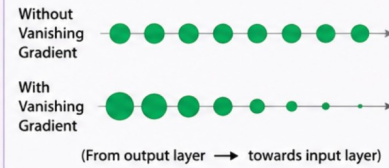


By the time the gradient reaches early layers, it is practically zero. Those weights barely update.

IMPACT

- Early layers learn extremely slowly (or not at all).
- Network fails to capture useful representations.
- Training becomes unstable and inefficient.

GRADIENT FLOW COMPARISON

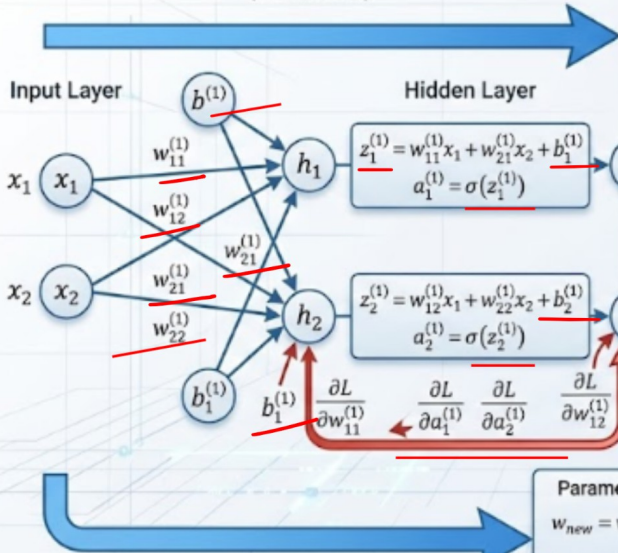


As gradients vanish, the network effectively becomes shallow — only the last few layers learn.

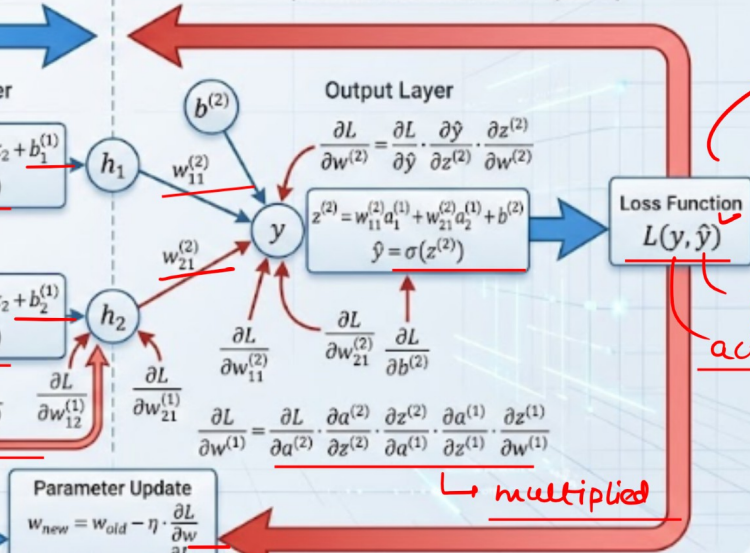
feedforward

Neural Network: Forward & Backward Propagation

FORWARD PROPAGATION (Prediction)



BACKWARD PROPAGATION (Gradient Calculation & Update)



Parameter Update
 $w_{new} = w_{old} - \eta \cdot \frac{\partial L}{\partial w}$
 $b_{new} = b_{old} - \eta \cdot \frac{\partial L}{\partial b}$

optimizer
(Error)

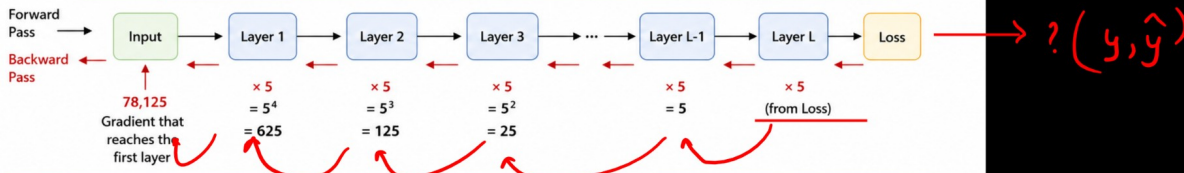
predicted
actual

multiplied

learning

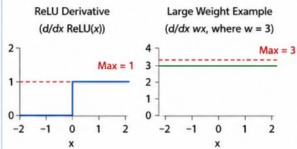
THE EXPLODING GRADIENT PROBLEM

As gradients backpropagate through many layers, they get multiplied by large numbers (> 1) repeatedly, growing exponentially. Early layers receive extremely large gradients, making training unstable.



WHY DOES IT HAPPEN?

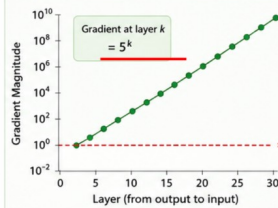
With certain activations (e.g., ReLU with large weights) or RNNs on long sequences, derivatives can be > 1 . Multiplying many such values causes the gradient to grow exponentially.



Derivatives (or effective local gradients) > 1 cause exponential growth when multiplied.

GRADIENT GROWS EXPONENTIALLY

Example: multiplying by 5 at each layer

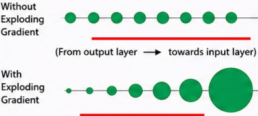


By the time the gradient reaches early layers, it becomes extremely large.

IMPACT

- Training becomes unstable (loss becomes NaN).
- Weights update too much and diverge.
- Model fails to learn anything useful.

GRADIENT FLOW COMPARISON



As gradients explode, updates become huge, causing divergence and preventing convergence.



HOW TO PREVENT?

Use proper weight initialization, normalization (BatchNorm), gradient clipping, smaller learning rate, or use gated architectures (LSTM/GRU) for RNNs.

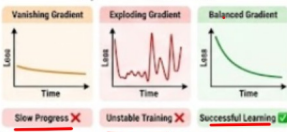
How Gradient Problems Affect Model Learning

Why Gradient Flow Directly Determines How Well Deep Neural Networks Learn

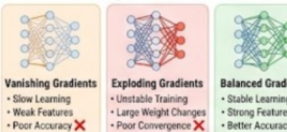
Gradient Quality Determines Learning Quality



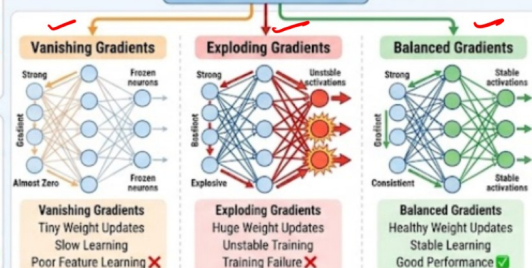
Impact on Loss Curves



Learning Outcome Dashboard



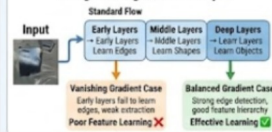
Deep Neural Network Training



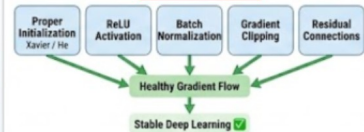
Impact on Weight Updates



Image Recognition Example



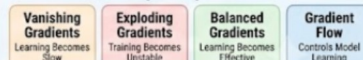
Modern Solutions



Cause-and-Effect Timeline



Key Concept Cards



Final Takeaway

Vanishing Gradients → Very Small Updates → Slow Learning ✗ Exploding Gradients → Very Large Updates → Unstable Training ✗ Balanced Gradients → Healthy Updates → Successful Deep Learning

Gradient quality directly affects how a neural network learns. Vanishing gradients slow learning, exploding gradients create instability, and balanced gradients allow deep networks to train effectively.

How can we avoid these issues?

- Overfitting
- vanishing/exploding gradient problem
- underfitting

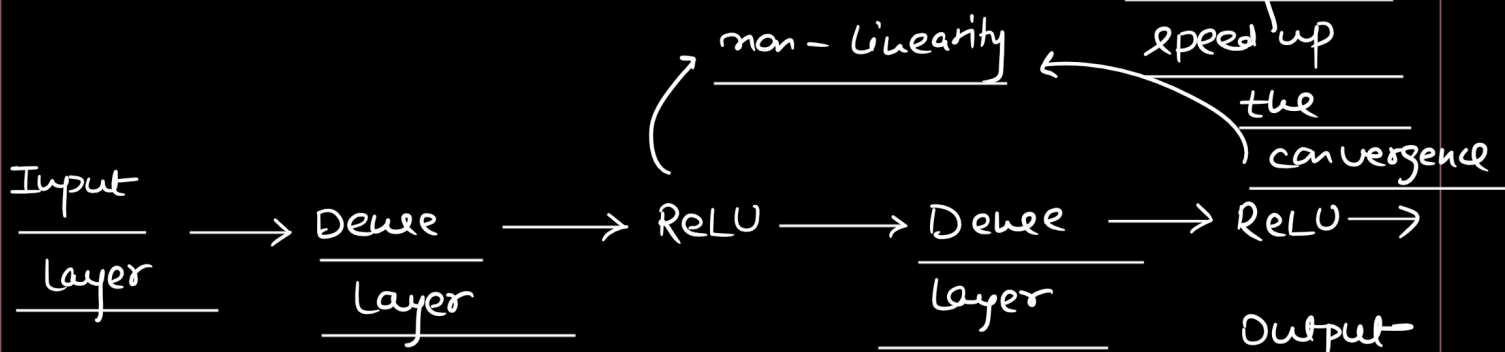
① Hidden layer → ReLU

② careful about
the num of
hidden layers
of hyperparameter tuning)

③ Regularization techniques (L1 & L2)

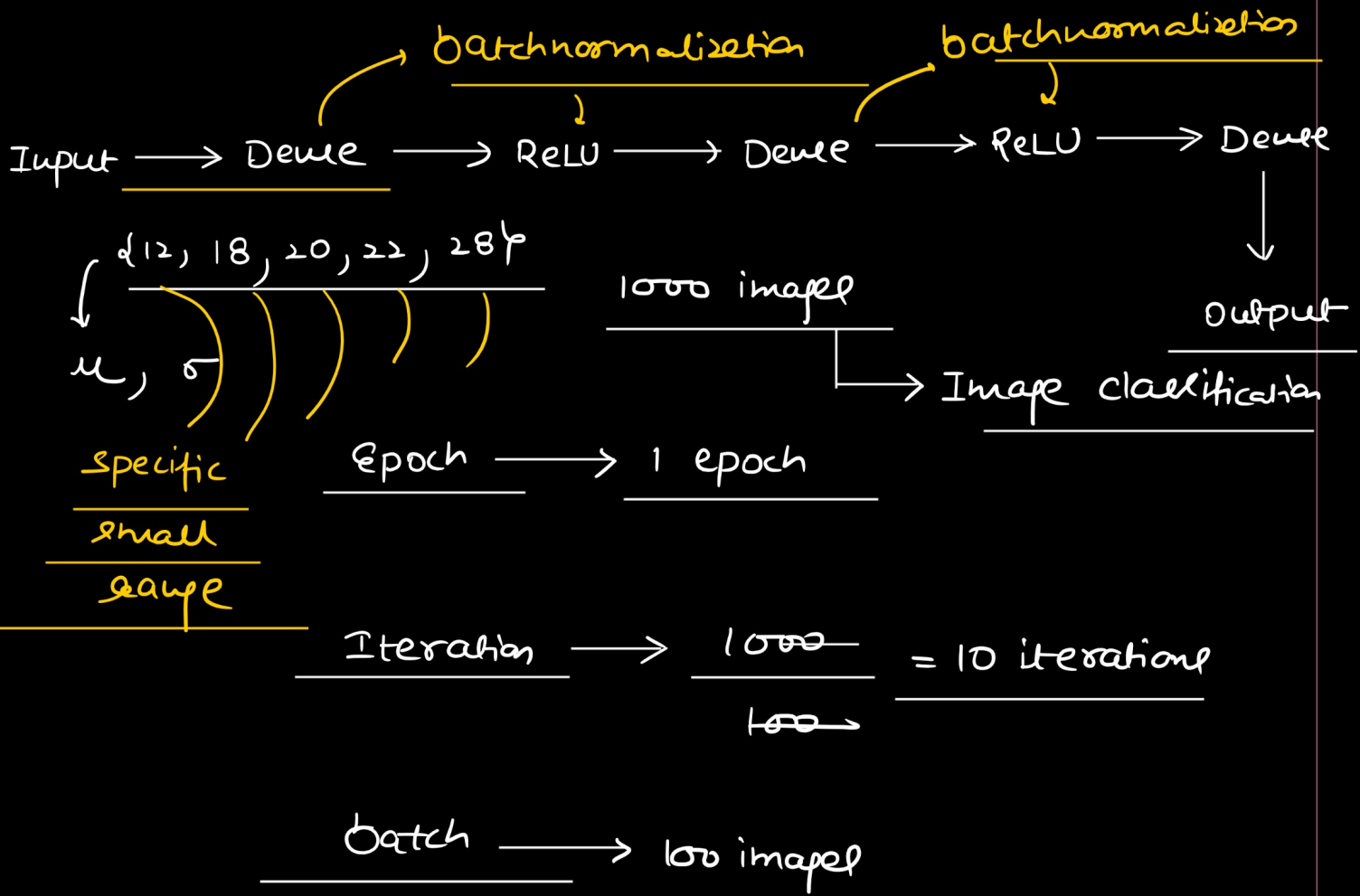
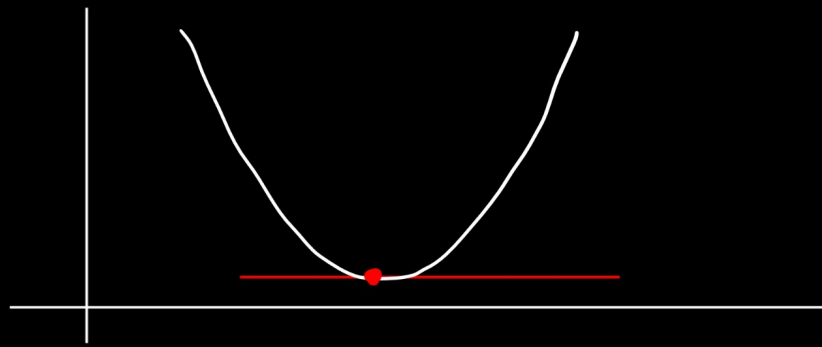
→ Dropout = 0.5

④ Batch Normalization → stabilize the
training &
speed up
the convergence



Loss function → extra term to

penalize the
weight values



Epoch = 100

overfitting

training accuracy

validation accuracy

Early

stopping

Accuracy →

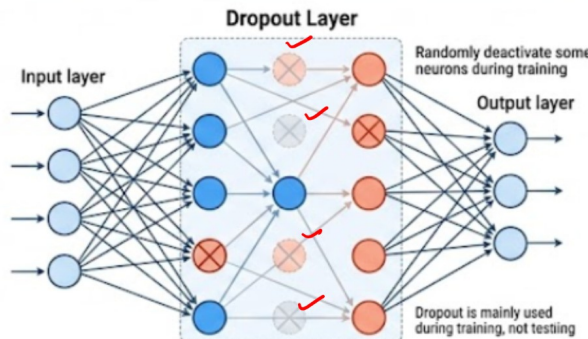
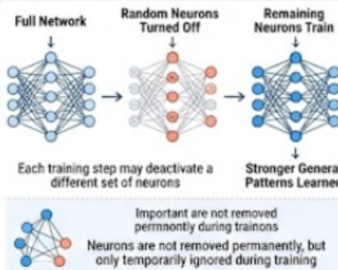
Epoch →

Dropout in Deep Networks

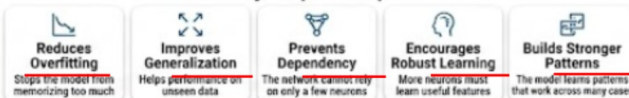
How randomly turning off neurons during training helps reduce overfitting and improve generalization

Overfitting Problem

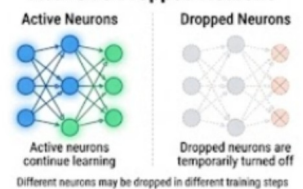
A model that performs very well on training / unseen data



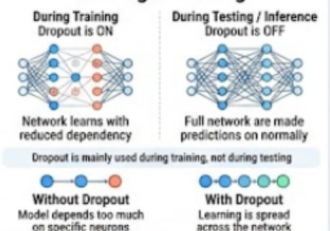
Why Dropout Helps



Active vs Dropped Neurons

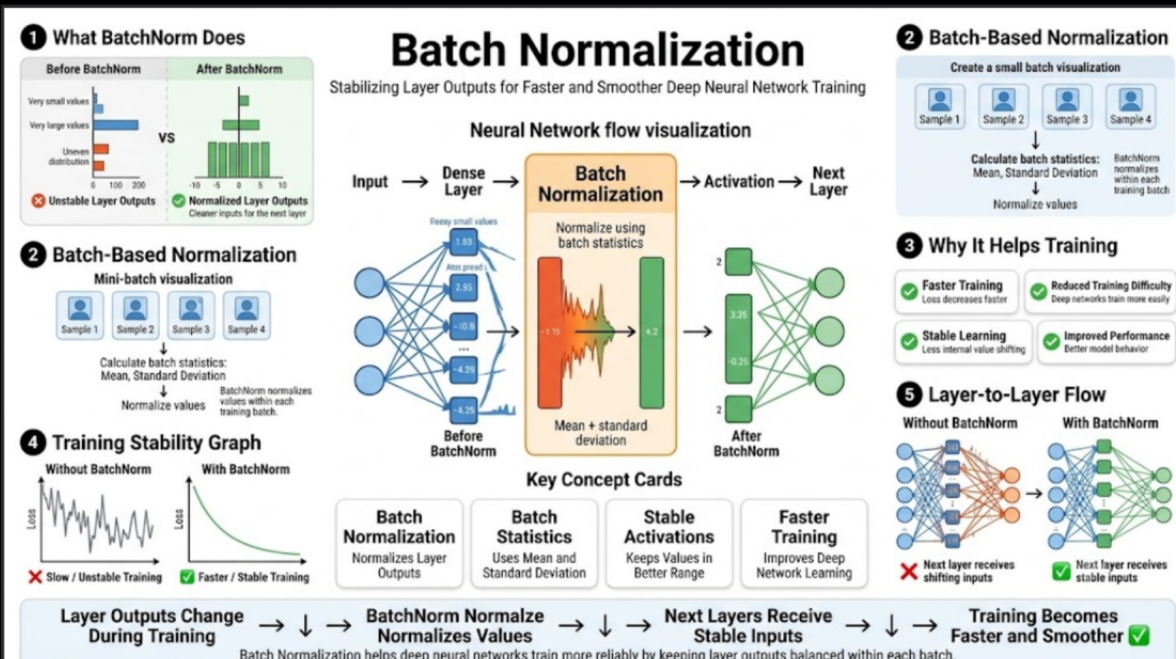


Training vs Testing



Deep Networks Can Overfit ↓ Dropout Randomly Turns Off Some Neurons During Training ↓ The Network Learns More Robust and General Patterns ↓ Overfitting Decreases and Generalization Improves ✓

Dropout is a regularization technique in deep learning that reduces overfitting by temporarily deactivating random neurons during training, helping the model learn stronger and more generalizable patterns



XOR Truth table

x_1	x_2	O/P
0	0	0
0	1	1
1	0	1
1	1	0

Non-linear data

